

Geospatial Analysis in Azure Databricks

Last updated by | Gupta Vishal kailash | Jul 23, 2019 at 7:42 AM GMT+1

Objective

The primary objective was to assess the capabilities of some of the most common geospatial libraries that are available to Databricks and make a recommendation as to which to use for the Telematics Phase II implementation.

The key spatial requirement for Telematics in this phase is to provide the necessary data for the existing Telematics 'on-site' and 'speeding on-site' Power BI reports.

To achieve this we need to process approximately 150,000 vehicle position polls that we receive each day from CTrack in CSV format and attempt to match these to the approximate 7,000 Anglian Water site polygon boundaries mastered by the AW Maps & Data team. This matching is achieved via a spatial join for which performance has often been a problem in the past. The site shape includes all types of Anglian Water site boundaries; including water recycling, treatment, pumping, private pumping and office locations.

Test details

Source Data

The data used for this testing consists of an anonymised sample of 36K polls taken from the actual CTrack Vehicle Positions daily feed to provide the points, and an AW site shape file already supplied by the Maps and Data team to provide the polygons. This data was manually uploaded to Databricks as the key requirement here is to test only the spatial join performance and capability of each library.

Environment

The table below details the version of the libraries and clusters configuration. There are a couple of points to notice:

- Magellan does not support Apache Spark 2.3+.
- The Magellan library 1.0.6 is under development but with no current firm release date. This release may solve the buffer issue described below

Library	Version	Runtime Version	Cluster Specification
Magellan	1.0.5	3.5 LTS (includes Apache Spark 2.2.1, Scala 2.11)	Standard_DS3_v2 driver type with 14GB Memory, 4 Cores and auto scaling enabled
GeoSpark/ GeoSparkSQL	1.1.3 / 2.3-1.1.3	4.3 (includes Apache Spark 2.3.1, Scala 2.11)	Standard_DS3_v2 driver type with 14GB Memory, 4 Cores and auto scaling enabled

Of the other libraries that I am aware of, GeoMesa has been ruled out mainly because Adatis has already had trouble getting it to work in Databricks

Geotrellis is still a consideration although the documentation does not cover any 'buffer' capability, so has been ruled out for now.

Any other packages found to date are simply wrappers on top of the native language to provide support in another language and in doing so offer no further functionality or advantage beyond those packages already considered.

Code and performance

Below are the code comparisons of the Databricks cells for both packages

Magellan

```
%scala
```

```
//Import Libraries
import magellan._
import org.apache.spark.sql.magellan.dsl.expressions._
import org.apache.spark.sql.Row
import org.apache.spark.sql.Column
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._
import org.apache.spark.sql._
import sqlContext.implicits._
```

```
//import shapefile
val polygons = sqlContext.read
  .format("magellan")
  .option("mode", "DROPMALFORMED")
  .option("magellan.index", "true")
  .option("magellan.index.precision", "30")
  .load("/FileStore/tables/")
  .where($"valid" === "true" )
  .withColumn("index", $"polygon" index 30)
  .toDF().show()

//import points
val points = sqlContext.read
  .format("com.databricks.spark.csv")
  .option("mode", "DROPMALFORMED")
  .option("header", true)
  .option("inferSchema", true)
  .load("/FileStore/tables/AW_VehiclePositions_Spatial_Sample_20180702040149.csv")
  .withColumn("poll", point($"Longitude", $"Latitude"))

points.show(10,50)

//spatial join
%scala
val joinresults = points.join(polygons).where($"poll".within($"polygon"))`
```

GeoSparkSQL

```
%scala
```

```
//Import Libraries and config session
```

```
import org.datasyslab.geosparksql.utils.GeoSparkSQLRegistrar
import org.datasyslab.geosparksql.utils.Adapter
import org.datasyslab.geosparksql.UDF.UdfRegistrar
import org.datasyslab.geosparksql.UDT.UdtRegistrar
import org.apache.spark.serializer.KryoSerializer;

import org.apache.spark.sql.Session;
import org.apache.spark.sql.geosparksql.strategy.join.JoinQueryDetector
import org.apache.spark.sql.Row
import org.apache.spark.sql.DataFrame
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._
```

```
//Initiate Spark Session
```

```
var sparkSession = SparkSession.builder()
    .appName("Telematics")
    // Enable GeoSpark custom Kryo serializer
    .config("spark.serializer", classOf[KryoSerializer].getName)
    //config("spark.kryo.registrator", classOf[GeoSparkKryoRegistrator].getName)
    .getOrCreate()
```

```
//Register GeoSparkSQL
```

```
GeoSparkSQLRegistrar.registerAll(sparkSession)
```

```
import org.datasyslab.geospark.formatMapper._
import org.datasyslab.geospark.formatMapper.shapefileParser._
```

```
//Import shapefile as spatial RDD

import org.datasyslab.geospark.spatialRDD._
import com.vividsolutions.jts.geom.Geometry

var spatialRDD = new SpatialRDD[Geometry]

spatialRDD.rawSpatialRDD = ShapefileReader.readToGeometryRDD(sparkSession.sparkContext, "/FileStore/tables/")

var rawSpatialDf = Adapter.toDf(spatialRDD,sparkSession)

rawSpatialDf.createOrReplaceTempView("rawSpatialDf")

var spatialDf = sparkSession.sql("""
    |SELECT  ST_GeomFromWKT(rddshape)
    | FROM rawSpatialDf --LIMIT 5
    """).stripMargin)
spatialDf.show(20,50)//.count()


//import points

val points = sqlContext.read
    .format("com.databricks.spark.csv")
    .option("mode", "DROPMALFORMED")
    .option("header", true)
    .option("inferSchema", true)
    .load("/FileStore/tables/AW_VehiclePositions_Spatial_Sample_20180702040149.csv").toDF()

//points.show(20,50)

points.createOrReplaceTempView("rawPointDf")
```

```
//spatial join
```

```
var polygonGeomWithin = sparkSession.sql(
  """
  | SELECT *
  | FROM rawSpatialDf sh , rawPointDf Pt
  | WHERE ST_Within(ST_Point(CAST(Pt.Longitude AS Decimal(24,20))), CAST(Pt.Lattitude AS Decimal(24,20))), ST_GeomFromWKT(sh.rd
  """)

polygonGeomWithin.count()//.show(20,50)
```



Performance Results

Package	Point Count	Polygon Count	Point to Site Matches	Average Processing Time (seconds)
Magellan*	35,966	6,863	6,608	7.41
GeoSparkSQL	35,966	6,863	6,608	9.63

*various index precision values were tested using the `SqlContextRead.options` shown below. In practice, this made very little difference to performance, with values of 5,30,50 and 100 being used (30 being the default). The package Documentation suggests that indexing only becomes beneficial when shape volumes are in excess of 100K and we are only dealing with approximately 7K.

```
.option("magellan.index", "true")
.option("magellan.index.precision", "30")
```

Issues

On the first attempt the spatial join did not return any results from the geospatial join using either package. Magellan stores the shape data as a geohash so it was not immediately obvious that the shape file projection was wrong, as it was using British National Grid (BNG) rather than the WGS84 projection required. Luckily Magellan provide a projection transform capability and once converted the matches were working as expected.

In terms of performance, there is very little to choose between the two, and although the SQL ST function syntax is more familiar I am inclined to use the Magellan library as the syntax is simpler.

The next and more substantial issue is that a suitable *buffer* function is not currently available in either library. GeoSparkSQL has something similar to *buffer* using the Circle function, in that a positive radius can be applied to a point, but unlike SQL Server it cannot apply a negative value to a shape in order to reduce its size by a specified number of degrees or meters.

Conclusion

It was encouraging that both methods returned the same results with very little difference in performance.

The disappointing part was the inability to apply a negative buffer to a shape. This is an important requirement in order to provide a mechanism in which to reduce the size of the site shape boundaries by a given number of meters (which may have been site specific) in order to reduce the amount of false matches, for example this might occur when a vehicle is travelling along a boundary road and the GPS signal drifts slightly and crosses the site boundary which has the potential to falsely report the poll as speeding on site.

We need to be careful not to reduce the shape size too much, as it could have a negative impact, if for example there is a parking space on the boundary inside a site, it could look like the driver has left the site.

1. Load the Site data in its original form without any shape reduction and deal with the worst case sites and re-work their shape data in conjunction with the maps and data team to create bespoke site boundaries on a case by case basis.
This option was immediately ruled out as during our previous POC we saw a number of real world examples of false matches due to non-optimal shape definitions.
2. Use the CTrack defined polygons defined. This is a function of CTrack online where FLMs can define their own Geozones and apply them as alerts, either to a particular vehicles or group of vehicles. This data is already ingested into DW via the Geozone flat file, but this has been ruled out because not all shapes are defined and those that are are not always well formed. The preference is to try and use Mast data instead.

e.g. example of poor boundary definition as defined in CTrack online, in this case the boundary includes public roads



3. Because the site data changes so infrequently, We can manually load the data in SQL server and apply an arbitrary shape reduction of 5 meters.
We would however need to check the results and optionally exclude reducing those already small sites, such as pumping stations, that would

otherwise make the reduced size so small as to be worthless; this can be achieved either by identifying pumping stations via the last 2 digits of the site code, or by applying the reduction to only those sites exceeding a to be agreed minimum area in square meters (the area being already available in the shape metadata).

Option 3 is Richard and I's preferred approach in that it will allow us to deliver something more quickly during this phase (including finding any further shortcomings with the shape definitions), With the aim being to liaise with the maps and data team as part of a future phase of work to customise the shapes for vehicle on-site reporting, as well agree a means to automate the ingestion of shape file updates. For example, one area of optimisation could be to split sites up with different speeding zones and to store that speed data as metadata within the shape file itself.

Recommendations

In terms of DevOps tasks I recommend we plan in the following work as part of phase ii

1. Liaise with Platform Engineer to ensure that a suitably versioned cluster with the required libraries pre-installed is available as part of the CD process in each environment.
2. Request an up to date shape file from the Maps & Data team containing all AW sites in the required WGS84 projection, if they can also apply the 5 Meter shape reduction to only those sites with a surface area greater than a yet to be agreed threshold they should do so, otherwise as a tactical solution the Dev squad can do this manually using a local SQL server instance.
3. Create an Ingestion pipeline that will ingest the shape file from an on-site folder, automatically unzip it, extract the .shp file and load it and its metadata into a data-frame that is then saved to an appropriate file format (Avro or Parquet?) in the BASE layer.
4. Using the Magellan Spark Library, create a data processing pipeline/notebook for a factSite object. this should includes the shape polygon (in Magellan hash format) and appropriate shape metadata, this ideally will become the conformed site dimension that will be combined with SAP Site master data in the future, but for now the basic shape metadata such as Site name and address should be sufficient.
5. Ensure that the FactEvent data processing pipeline includes as part of its notebook the necessary logic to do a spatial join to the site dim and insert the dimSITE SID as a Foreign Key to link any given poll to its linked site, or to default to -1 SID to indicate that the position was not matched to any particular site.
6. The downstream processing from here into the Power BI reports has already been catered for in other user-stories.

sources

<https://github.com/harsha2010/magellan> 

<http://datasystemslab.github.io/GeoSpark/> 

30/01/2019 Update

This topic was discussed on the AWBI Phase 2 - Technical Design Forum and it was agreed to

1. Use GeoSparkSQL over Magellan, simply because the MIT origin and later Spark version suggest better longer term maintenance and therefore viability.
2. Agreed to do spatial analysis in processing layer only and decouple the ingestion layer to store the shape file in an agnostic format e.g. line strings etc
3. A dedicated cluster will be stood up in the data processing layer to host the library and do the geospatial processing.